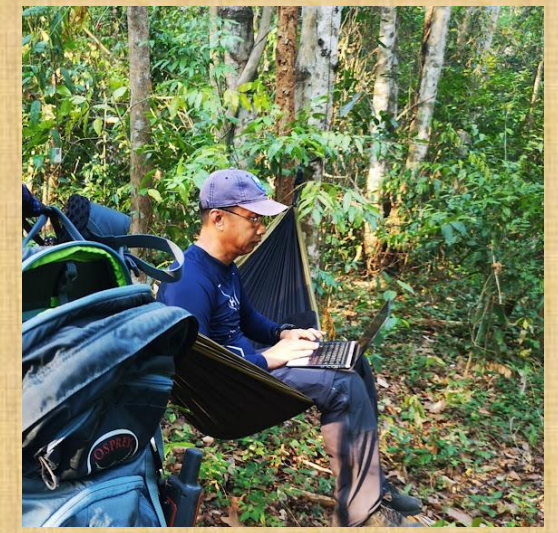
ITCS214 : DATA STRUCTURES & ALGORITHM ANALYSIS



Lecture 8 Recursion & Algorithms



Asst. Prof. Dr. Ananta Srisuphab
ananta.sri@mahidol.ac.th
2026/1

Mahidol University
Faculty of Information and Communication Technology

Where we start, where we finish

YOU ALREADY KNOW

- Writing and calling Java methods with parameters and return values
- Loops (for / while) and arrays
- Basic Big-O: O(1), O(n), O(n^2) and what they compare
- Reading a stack trace when a program crashes

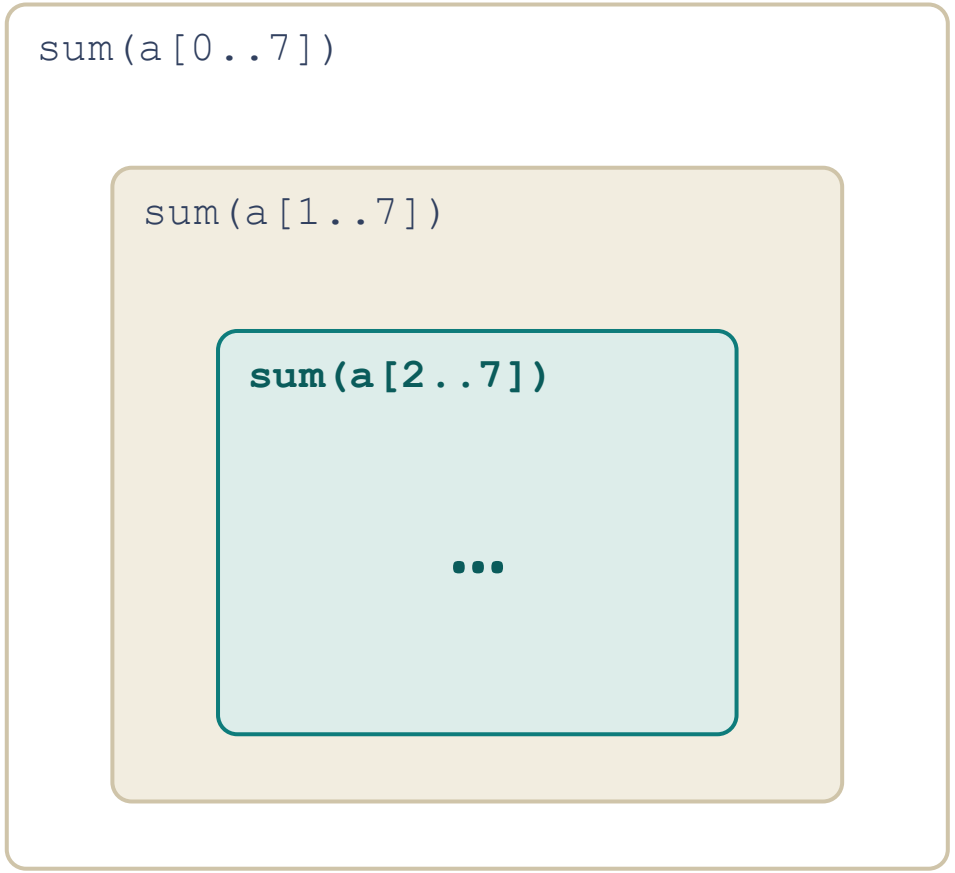
BY 3:00 YOU CAN

- Design a recursive method from a base case + smaller subproblem
- Trace the call stack and predict output and return values
- Argue correctness with induction, and termination separately
- Derive and solve recurrences: O(n), O(log n), O(n log n)
- Speed up naive recursion with memoization; know when to go iterative
- Recurse over trees and write a first backtracking search

What recursion is (before any code)

A recursive method solves a problem by calling itself on a **strictly smaller** instance of the same problem.

- Same shape, smaller size: sum of n numbers = first + sum of the remaining n−1
- Everyday versions: folders inside folders, boxes inside boxes
- The chain must bottom out at a case so small we can answer it directly
- *Recursion is a design strategy first; the code is just its transcription.*



The recursive contract: three promises

1

Base case

Answer the smallest input(s) directly — no recursive call at all.

2

Progress

Every recursive call moves strictly closer to a base case.

3

Trust

Assume the recursive call returns the right answer for its smaller input — then use it. (The “recursive leap of faith.”)

Break any one promise and the recursion is wrong — we will see how each failure looks at runtime.

Termination: a quantity that must run out

A recursion terminates when some non-negative measure **strictly decreases** on every call.
It cannot decrease forever — so the calls must stop.

Problem	Measure that shrinks	Stops at
`factorial(n)`	n	`n == 0`
`binarySearch(lo,hi)`	range size hi−lo+1	`lo > hi`
`tree size(t)`	nodes in subtree	`t == null`

SPOT THE TRAP

```
static int bad(int n) {
    if (n == 0) return 0;
    return bad(n - 2);
}
```

- bad(6) → 4 → 2 → 0 ✓ stops
- bad(7) → 5 → 3 → 1 → −1 → … never hits 0
- The measure must shrink toward a case you actually reach

Factorial: the definition IS the design

MATH DEFINITION

$$0! = 1$$
$$n! = n \cdot (n-1)! \quad \text{for } n \geq 1$$

READ IT AS A RECIPE

- Base case given: 0! is 1, no work needed
- Recursive case given: n! in terms of (n−1)!
- Progress given: n−1 is strictly smaller

Check the contract before writing any Java

- Promise 1 (base): n = 0 answered directly. Promise 2 (progress): n → n−1 → … → 0. Promise 3 (trust): assume (n−1)! comes back correct, multiply by n.
- *Every recursive design session should end with this three-line audit — make it a habit.*

factorial in Java, with its assumptions

RECURSIONLECTURE.JAVA

```java
/** Returns n!. Assumes 0 <= n <= 20 (21! overflows long). */
static long factorial(int n) {
    if (n < 0)
        throw new IllegalArgumentException("n must be >= 0");
    if (n == 0) return 1L;          // base case
    return n * factorial(n - 1);  // recursive case
}
```

- State input assumptions in the doc comment — they are part of the method
- Reject inputs that break the contract (n < 0 would never terminate)
- long return type buys headroom: why 20?

Line ↔ promise

`if (n == 0)`

base case

`n - 1`

progress

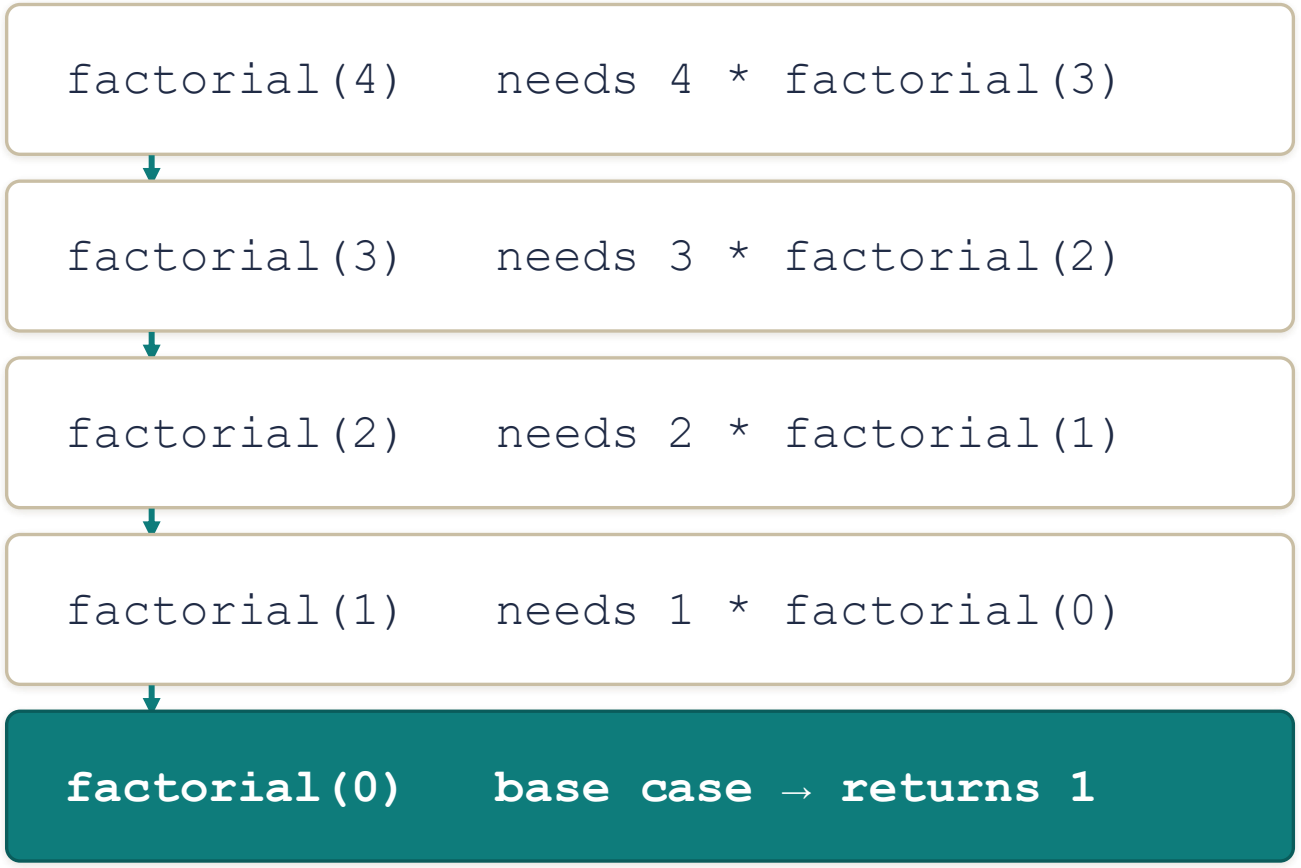
`factorial(n-1)`

trust: it returns (n−1)!

`n * …`

combine into n!

Tracing factorial(4): calls stack up

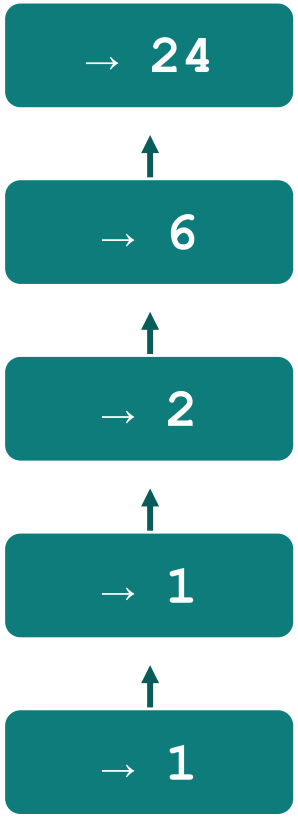


each call pushes a new frame ↓

- Every call gets its own frame: its own n, its own place to resume
- Frames above are paused mid-expression, waiting for a value
- Five frames are alive at once — this is the stack depth
- **Nothing has been multiplied yet!**

...then results cascade back up

```
factorial(4)    resumes: 4 * 6
factorial(3)    resumes: 3 * 2
factorial(2)    resumes: 2 * 1
factorial(1)    resumes: 1 * 1
factorial(0)    → 1
```



returns pop frames ↑

- A return delivers a value to the frame below and pops
- Each paused multiplication finally runs, bottom-up
- 1 → 1 → 2 → 6 → 24: the answer assembles on the way back

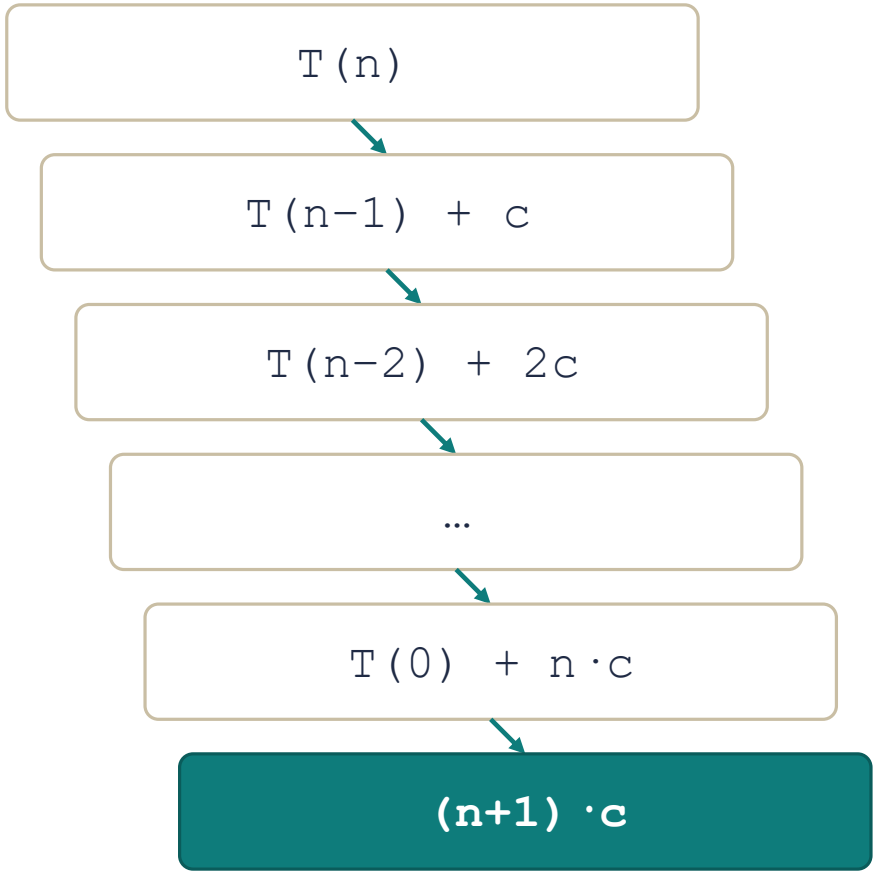
Cost as a recurrence: T(n) for factorial

$$T(n) = T(n-1) + c$$
$$T(0) = c$$

"the cost of factorial(n) is the cost of factorial(n−1) plus a constant amount of local work"

Unroll it: T(n) = c + c + … + c (n+1 times) = **Θ(n)**

Stack depth: n+1 frames at the peak → Θ(n) space, too.



Why we may “trust the call”: induction

MATHEMATICAL INDUCTION

- **To prove P(n) for all n ≥ 0:**
- Base: prove P(0) directly
- Step: assume P(k−1) — the induction hypothesis — and derive P(k)
- *Conclusion: P(0), P(1), P(2), … all hold: dominoes.*

RECURSION, MIRRORED

- Base case in code ↔ base of the induction
- Recursive call on n−1 ↔ induction hypothesis
- Code after the call ↔ the derivation step
- *The “leap of faith” is the induction hypothesis wearing a hoodie.*

A three-line proof that factorial works

Claim P(n): factorial(n) returns n! (for every n ≥ 0 within long range)

Base P(0): the code hits “if (n == 0) return 1L” — and 0! = 1. ✓

Step Assume P(n−1): the call factorial(n−1) returns (n−1)!. The code returns n · (n−1)! = n!. ✓

Done By induction, P(n) holds for every n ≥ 0. The “trust” was never faith — it was the hypothesis.

Correctness ≠ termination. This proof shows the value is right IF the calls stop. The decreasing measure n → n−1 → … → 0 shows they do. Two arguments, both required — next slide.

Two separate obligations

PARTIAL CORRECTNESS

- **“IF it returns, the value is right.”**
- Tool: induction on input size
- Can hold even for code that loops forever on some inputs
- *bad(7) never lies — it never answers.*

TERMINATION

- **“It DOES return, for every valid input.”**
- Tool: a non-negative measure that strictly decreases each call
- factorial: n. binarySearch: hi−lo+1. trees: subtree size
- *No measure? Suspect the recursion.*

partial correctness + termination = total correctness

Java detail #1: overflow is silent

Type	Bits	Max value	Last n with n! in range
int	32	2,147,483,647	12! = 479,001,600
long	64	9,223,372,036,854,775,807	20! = 2,432,902,008,176,640,000

```
// tested in RecursionLecture.java:
21 * factorial(20)  ==  -4249290049419214848
// 21! wrapped around — no exception, no warning
```

- That is why factorial documents 0 ≤ n ≤ 20 — the limit is the type, not the algorithm
- Need bigger? java.math.BigInteger trades speed for unbounded range

Rule of thumb

Before you trust a numeric recursion, find the first input whose RESULT no longer fits the return type — and write that bound into the doc comment.

Java detail #2: the stack is finite

- Every active call holds one stack frame: parameters, locals, return address
- The JVM thread stack is fixed-size (commonly ~512 KB–1 MB; tune with -Xss)
- Depth beyond capacity → StackOverflowError — often tens of thousands of frames
- Java does not guarantee tail-call elimination: deep recursion stays deep

```
// remove the n < 0 guard, then:
factorial(-1);
Exception in thread "main"
    java.lang.StackOverflowError
  at RecursionLecture.factorial(...)  × thousands
```

Triage a StackOverflowError

- **1. Read the repeating frame in the trace — that is your recursion**
- **2. Check the contract: unreachable base case? no progress?**
- **3. If the recursion is CORRECT but deep (n ≈ 10^5+): rewrite iteratively or raise -Xss**
- *It is an Error, not an Exception: don't catch it — fix the cause.*

Binary search: discard half, repeat

- **Precondition: the array is sorted ascending — no order, no binary search**
- Compare the key with the middle element
- Equal → found. Key larger → live only in the right half. Smaller → left half
- Same problem, half the size: a recursion waiting to happen

2	5	8	12	**16**	23	38	56	72	91
0	1	2	3	4	5	6	7	8	9

key = 23: a[4] = 16 < 23 → everything left of index 5 is out

2	5	8	12	16	23	38	**56**	72	91

Invariant to carry

If the key is anywhere, it is inside a[lo..hi]. Every step keeps this true while shrinking hi−lo.

binarySearch in Java

RECURSIONLECTURE.JAVA

```
/** Index of key in sorted a[lo..hi], or -1 if absent.
 *  First call: binarySearch(a, key, 0, a.length - 1). */
static int binarySearch(int[] a, int key, int lo, int hi) {
    if (lo > hi) return -1;         // empty range: not found
    int mid = lo + (hi - lo) / 2;   // see note below!
    if (a[mid] == key) return mid;
    if (a[mid] < key)
        return binarySearch(a, key, mid + 1, hi);
    else
        return binarySearch(a, key, lo, mid - 1);
}
```

Contract audit

- **Base: lo > hi (empty)**
- **Progress: window loses ≥ half + 1 element (mid excluded)**
- **Trust: the half-search returns the right index**

Why not (lo + hi) / 2? For huge arrays lo + hi can overflow int and go negative. lo + (hi − lo) / 2 cannot. This exact bug sat in the JDK's own binary search for nine years.

Guided trace: key = 23




```
call 1: lo=0 hi=9 mid=4
16 < 23 → go right
```




```
call 2: lo=5 hi=9 mid=7
56 > 23 → go left
```




```
call 3: lo=5 hi=6 mid=5
a[5]=23 → return 5 ✓
```

Each return value travels back unchanged: call 3 returns 5 → call 2 returns 5 → call 1 returns 5. The recursion found it; the stack just delivers it.

$T(n) = T(n/2) + c \rightarrow \Theta(\log n)$

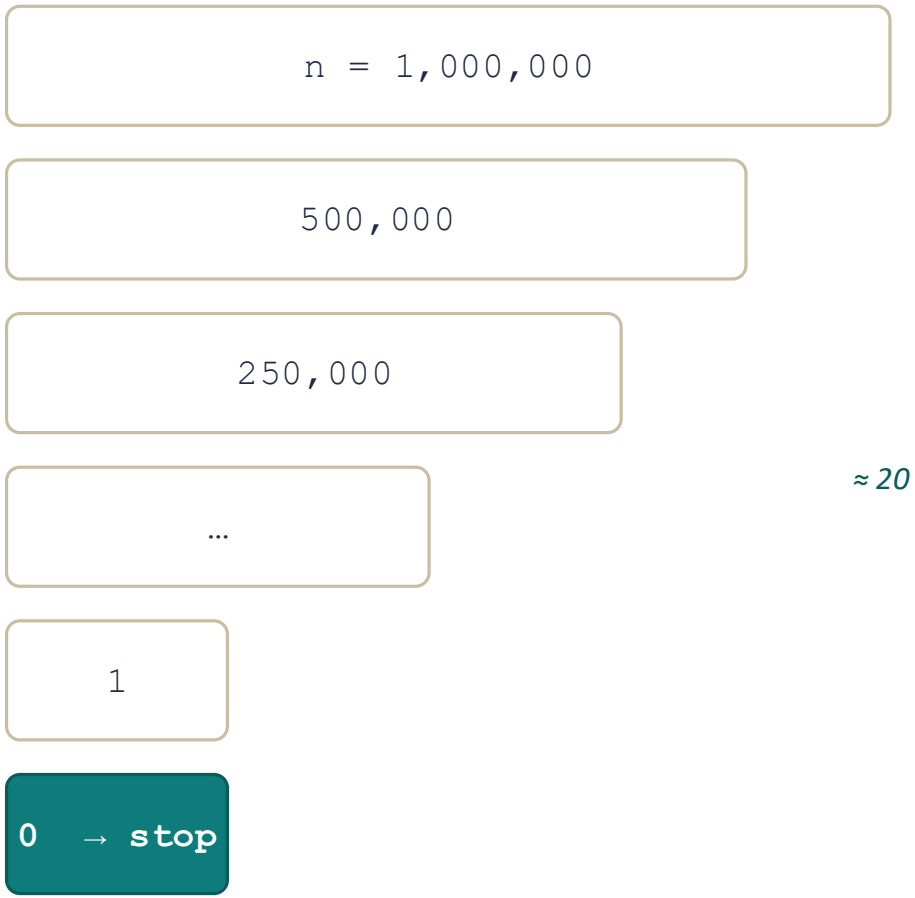
$$T(n) = T(n/2) + c$$
$$T(0) = c$$

constant work per call, one call on half the window

How many halvings until size ≤ 1?

$n \rightarrow n/2 \rightarrow n/4 \rightarrow \ldots \rightarrow 1$ after $\log_2 n$ steps

$T(n) = c \cdot (\log_2 n + 2) = \Theta(\log n)$



≈ 20 halvings

Total time ≠ peak stack depth

TOTAL TIME

Sum of the work done by EVERY call that ever runs — alive or finished.

"How long do I wait?"

PEAK STACK DEPTH

Longest chain of calls alive AT THE SAME MOMENT — frames stacked at the peak.

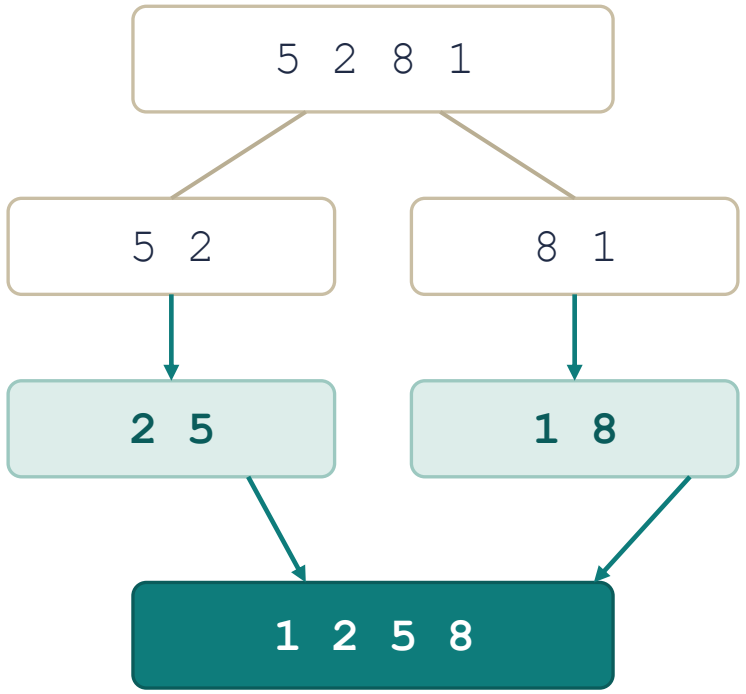
"Can it StackOverflow?"

Algorithm	Total time	Peak stack depth
`factorial(n)`	Θ(n)	Θ(n)
`binarySearch (n elems)`	Θ(log n)	Θ(log n)
`mergeSort (preview)`	Θ(n log n)	Θ(log n) — tiny!

Same words students blur together; different resources, different failure modes.

Divide & conquer: sort by splitting

- **Two ideas, kept separate:**
- Divide: a 1-element (or empty) run is already sorted — that is the base case
- Conquer: sort each half by trusting the recursion
- Combine: MERGE two sorted runs into one sorted run — the only real work



split ↓ sort halves merge ↑

sorting happens on the way back up

Merging two sorted piles

Face-up card piles: repeatedly take the smaller top card. Each comparison retires one element, so merging n elements costs Θ(n).

merge: two fingers, one pass

RECURSIONLECTURE.JAVA

```
/** Merges sorted runs a[lo..mid] and a[mid+1..hi]. */
static void merge(int[] a, int lo, int mid, int hi, int[] tmp) {
    for (int k = lo; k <= hi; k++) tmp[k] = a[k];   // snapshot
    int i = lo, j = mid + 1;
    for (int k = lo; k <= hi; k++) {
        if      (i > mid)          a[k] = tmp[j++]; // left done
        else if (j > hi)           a[k] = tmp[i++]; // right done
        else if (tmp[j] < tmp[i]) a[k] = tmp[j++]; // right smaller
        else                       a[k] = tmp[i++]; // tie → left
(stable)
    }
}
```

Read it as fingers

- **i walks the left run, j the right, k the output**
- Exactly one element retires per loop turn → Θ(hi−lo+1)
- Ties take LEFT first → equal keys keep their order (stability)

No recursion in merge itself — it is the honest Θ(n) worker that the recursion organizes.

mergeSort: the recursion around merge

RECURSIONLECTURE.JAVA

```
/** Sorts a ascending. */
static void mergeSort(int[] a) {
    mergeSort(a, 0, a.length - 1, new int[a.length]);
}

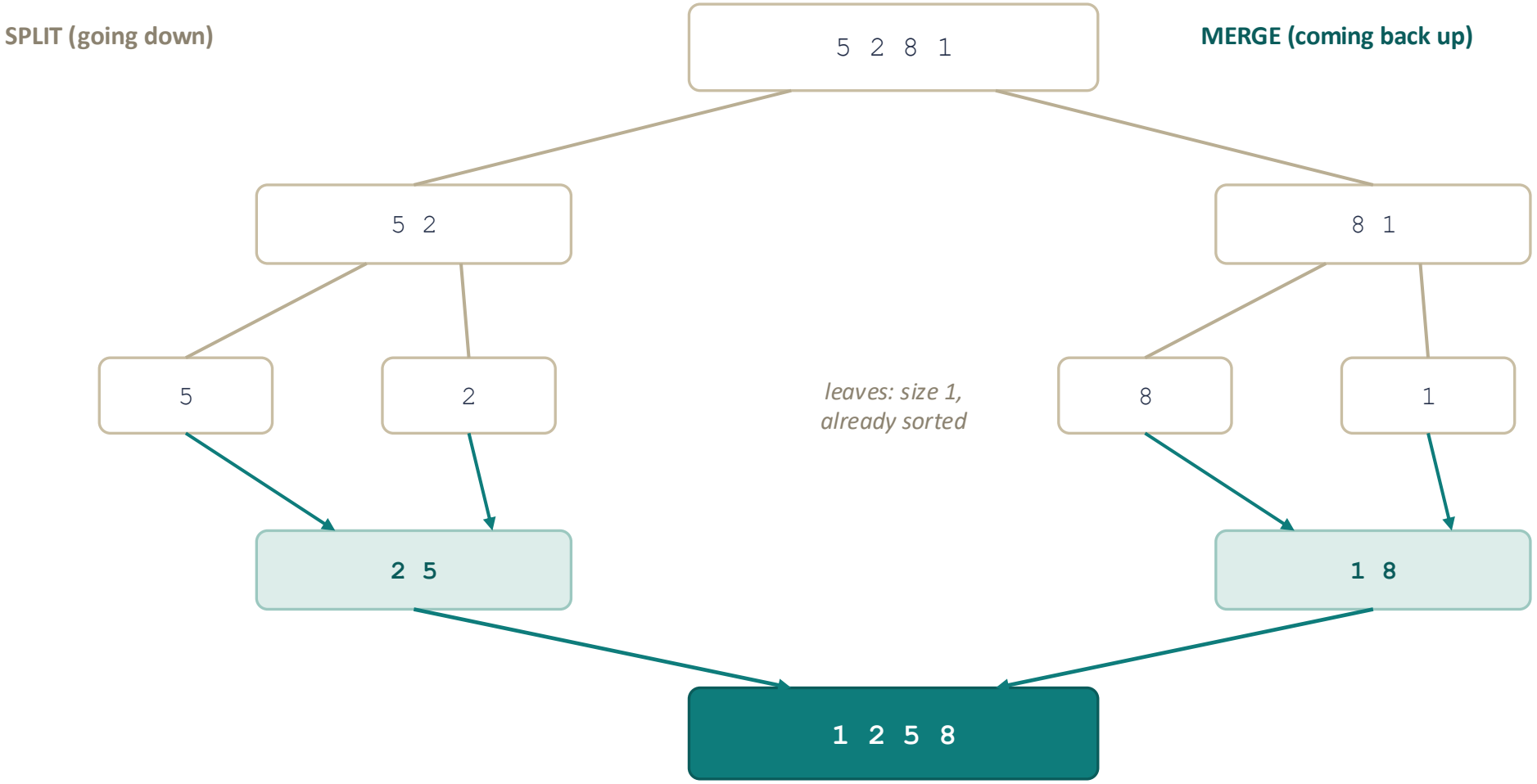
static void mergeSort(int[] a, int lo, int hi, int[] tmp) {
    if (lo >= hi) return;              // 0 or 1 element: sorted
    int mid = lo + (hi - lo) / 2;
    mergeSort(a, lo, mid, tmp);        // trust: left sorted
    mergeSort(a, mid + 1, hi, tmp);    // trust: right sorted
    merge(a, lo, mid, hi, tmp);        // combine
}
```

Design notes

- Public one-arg wrapper hides lo/hi/tmp — a common recursion pattern
- tmp allocated ONCE, reused by every merge
- TWO recursive calls: our first branching recursion

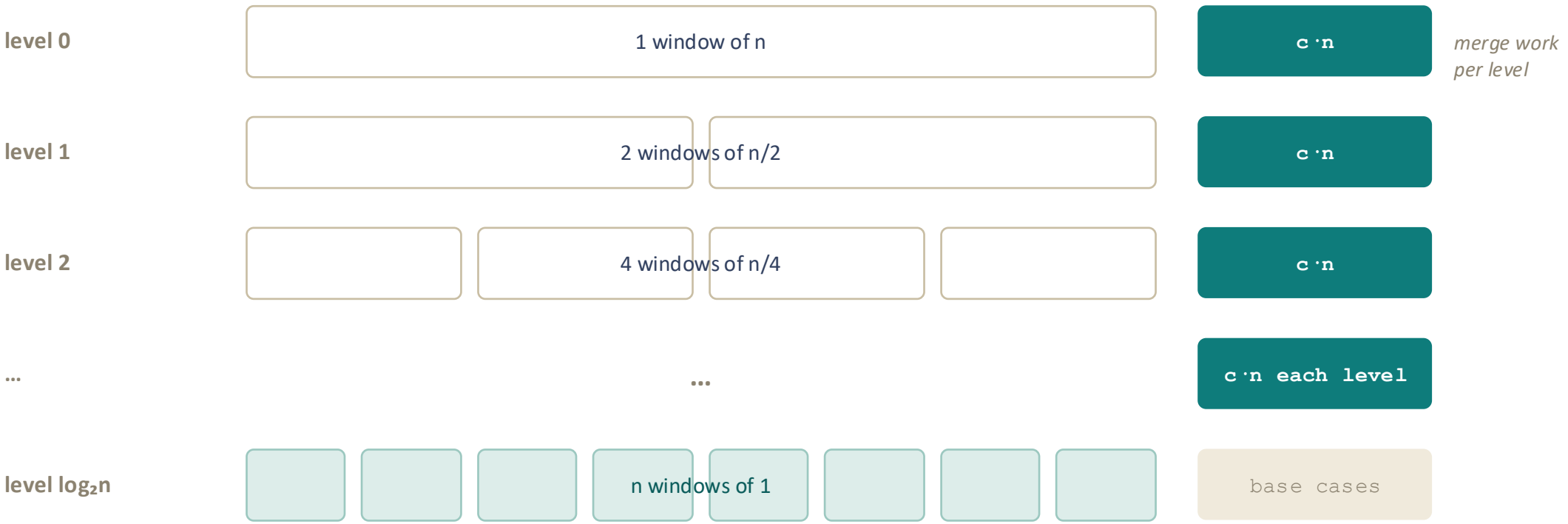
`Base case lo >= hi, not lo == hi:` an empty array calls mergeSort(a, 0, -1) — with == it would recurse on nonsense windows. Cheap robustness.

The whole story for [5, 2, 8, 1]



order of events: left subtree finishes COMPLETELY before the right one starts

$T(n) = 2 \cdot T(n/2) + c \cdot n \rightarrow \Theta(n \log n)$



$\log_2 n + 1$ levels × c·n merge work per level = **$\Theta(n \log n)$** — every level touches every element exactly once.

Fibonacci: an innocent-looking recursion

DEFINITION

$$fib(0) = 0,\quad fib(1) = 1$$
$$fib(n) = fib(n-1) + fib(n-2)$$

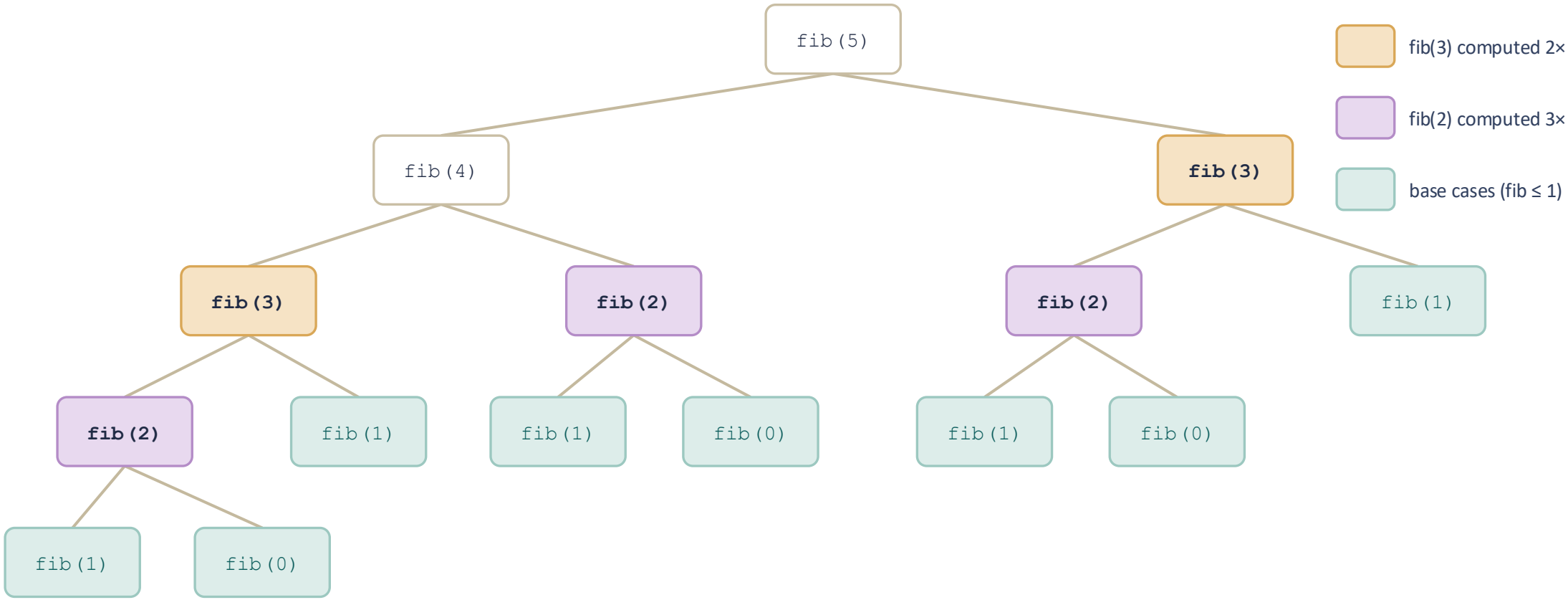
TRANSCRIBED DIRECTLY

```
/** Assumes 0 <= n <= 92
 *  (fib(93) overflows long). */
static long fibNaive(int n) {
    if (n <= 1) return n;
    return fibNaive(n - 1)
         + fibNaive(n - 2);
}
```

Audit says: correct. Clock says: help.

- Base ✓ ($n \le 1$), progress ✓ ($n-1$, $n-2$ with $n \ge 2$), trust ✓
- Induction proves it returns fib(n) — the VALUE is beyond doubt
- Yet fibNaive(50) takes minutes while fibNaive(30) is instant
- *New lesson: a recursion can be perfectly correct and computationally disastrous. Correctness and cost are separate audits.*

The tree fibNaive(5) actually builds



15 calls to compute one number — and the same subtrees, rebuilt from scratch.

Measuring the explosion

n	fibNaive(n) total calls	feel
10	177	instant
20	21,891	instant
30	2,692,537	blink
40	331,160,281	seconds
50	≈ 4.1 × 10^{10}	many minutes

call counts measured live by fibCallCount() in RecursionLecture.java

Why ~1.6× per step?

C(n) = C(n−1) + C(n−2) + 1

The call count obeys Fibonacci's own recurrence — so it grows like fib itself: $\Theta(\phi^n)$, $\phi \approx 1.618$. Each +1 to n multiplies the work by the golden ratio.

Diagnosis, not despair: the waste is only REPEATED subproblems. There are just n+1 distinct values fib(0..n). Compute each once and the explosion vanishes.

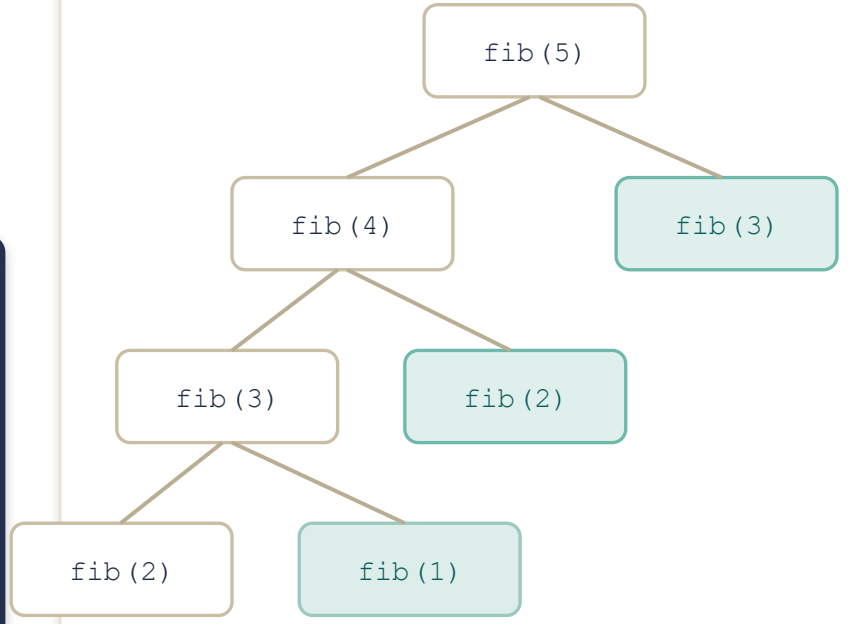
Memoization: pay for each subproblem once

Before computing, check a cache.
After computing, fill the cache.
Same recursion — plus a memory.

WRAPPER PATTERN AGAIN

```
static long fibMemo(int n) {
    long[] memo = new long[n + 1];
    Arrays.fill(memo, -1L);   // -1 = "not yet"
    return fibMemo(n, memo);
}
static long fibMemo(int n, long[] memo) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];   // hit!
    memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
    return memo[n];
}
```

The tree, pruned



green = cache hit: returns instantly, no subtree built beneath it

Three Fibonaccis, one honest table

	fibNaive	fibMemo	fibIter
Total time	$\Theta(\varphi^n)$	$\Theta(n)$	$\Theta(n)$
Peak stack depth	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Extra heap memory	—	$\Theta(n)$ cache	$\Theta(1)$: two longs
Reads like the math?	perfectly	mostly	not really
fib(50) wall-clock	minutes	microseconds	microseconds

Choosing: start with the clearest correct recursion → memoize if subproblems overlap → go iterative when depth or constant factors actually hurt. Measure before step 3.

PART 8 · CLOSE

180 minutes, six ideas

1 The contract

Base case · strict progress · trust the smaller call. Audit every recursion against all three.

2 Induction = trust, licensed

Correctness by induction; termination by a shrinking measure. Two proofs, both owed.

3 The stack is real

Frames push and pop; depth ≠ total calls; long limits and StackOverflowError are part of the design.

4 Recurrences price the code

$T(n-1)+c \to \Theta(n)$ · $T(n/2)+c \to \Theta(\log n)$ · $2T(n/2)+cn \to \Theta(n \log n)$ · $T(e/2)+c \to \Theta(\log e)$

5 Overlap → memoize

Naive fib repeats subtrees: $\Theta(\phi^n)$. Cache each distinct subproblem once: $\Theta(n)$. Iterate when depth hurts.

6 Structure guides recursion

Trees recurse on subtrees; backtracking = choose, explore, unchoose — and always undo.